

4.3 Smart Access Control System

Smart Access Control System

Overview

This tutorial builds a smart access control system using ESP32, featuring dual authentication methods: RFID card (RC522) and password keypad. The system includes OLED display for password input feedback, servo motor for door locking/unlocking, and buzzer for authentication sound feedback.

Component Required

- (1) x Elegoo ESP32
- (1) x 128×64 OLED Display (SSD1306)
- (1) x RC522 RFID Module
- (1) x 4×4 Matrix Keypad
- (1) x Servo Motor (SG90)
- (1) x Active Buzzer

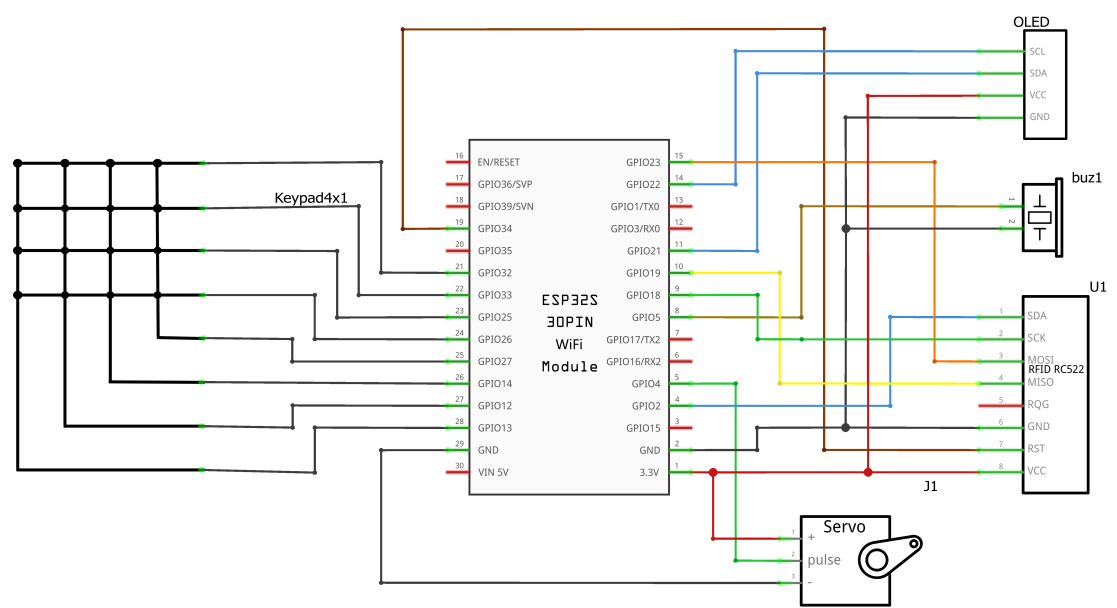
Connection Schematic

Component	ESP32 Pin	Description
OLED VCC	3.3V	OLED power
OLED GND	GND	OLED ground
OLED SDA	GPIO 21	I2C data line
OLED SCL	GPIO 22	I2C clock line
RC522 VCC	3.3V	RC522 power
RC522 GND	GND	RC522 ground
RC522 RST	GPIO 34	RC522 reset
RC522 SDA(SS)	GPIO 2	SPI chip select
RC522 MOSI	GPIO 23	SPI data output
RC522 MISO	GPIO 19	SPI data input
RC522 SCK	GPIO 18	SPI clock
Servo Signal	GPIO 4	Servo PWM control
Buzzer	GPIO 5	Active buzzer control
Keypad Row 0	GPIO 13	Row pin 0
Keypad Row 1	GPIO 12	Row pin 1
Keypad Row 2	GPIO 14	Row pin 2
Keypad Row 3	GPIO 27	Row pin 3
Keypad Col 0	GPIO 26	Column pin 0
Keypad Col 1	GPIO 25	Column pin 1

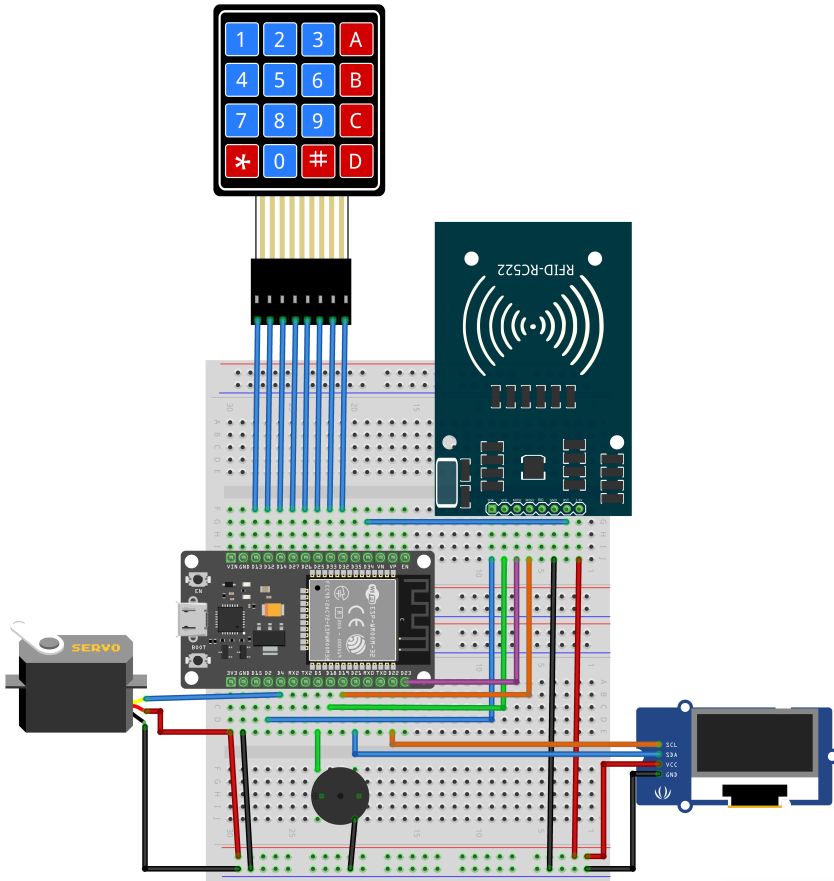
Component	ESP32 Pin	Description
Keypad Col 2	GPIO 33	Column pin 2
Keypad Col 3	GPIO 32	Column pin 3

[Back to Elegoo](#)

Connection Schematic



Wiring diagram



Code

Program Architecture

```
1 | Smart_Access_Control.ino (Main Program)
2 |   |—— Calls → RC522 Module
3 |   |   |—— RC522_control.h (Pin Definitions, Servo Config, Function Declarations)
4 |   |   |—— RC522_control.cpp (RFID Initialization, Authorization, Door Control)
5 |   |—— Calls → Keypad & OLED Module
6 |   |   |—— key.h (OLED Config, Function Declarations)
7 |   |   |—— key.cpp (Keypad Input, Password Verification, OLED Display)
```

Code Explanation

1. Header File: RC522_control.h

Function: Define RC522 pin macros, servo configuration parameters, authorized UID declarations, and function interfaces.

Pin Macro Definitions:

```
1 | #define RST_PIN 34 // RC522 reset pin
2 | #define SS_PIN 2 // RC522 chip select pin (SPI SS)
3 | #define SERVO_PIN 4 // Servo motor signal pin (PWM compatible)
```

- `RST_PIN` : RC522 module reset pin (GPIO 34)
- `SS_PIN` : RC522 chip select pin (GPIO 2, SPI Slave Select)
- `SERVO_PIN` : Servo motor signal pin (GPIO 4, PWM compatible)

Servo Configuration Macros:

```
1 | #define LOCKED_ANGLE 0 // Door locked angle (degrees)
2 | #define UNLOCKED_ANGLE 30 // Door unlocked angle (degrees)
3 | #define UNLOCK_DURATION 3000 // Unlock hold duration (milliseconds, 3 seconds)
```

- `LOCKED_ANGLE` : Door locked angle (0 degrees)
- `UNLOCKED_ANGLE` : Door unlocked angle (30 degrees)
- `UNLOCK_DURATION` : Unlock hold duration (3000 milliseconds, i.e., 3 seconds)

Authorized UID Configuration:

```
1 | extern const byte authorizedUIDs[][4]; // External declaration: List of authorized card UIDs
2 | extern const int AUTHORIZED_COUNT; // External declaration: Number of authorized cards
```

- `authorizedUIDs` : List of authorized card UIDs (defined in .cpp)
- `AUTHORIZED_COUNT` : Number of authorized cards (automatically calculated)

Global Object Declarations:

```
1 | extern MFRC522 mfr522; // External declaration: RC522 instance
2 | extern MFRC522::MIFARE_Key key; // External declaration: MIFARE key instance
3 | extern Servo doorServo; // External declaration: Servo motor instance
```

- `mfr522` : RC522 module object
- `key` : MIFARE card key object (used for authentication)
- `doorServo` : Servo motor object (controls door lock)

Function Declarations:

```
1 | void initRC522(); // Initialize RC522 RFID reader
2 | void initServo(); // Initialize servo motor
3 | void initMifareKey(); // Initialize MIFARE encryption key
```

```

4 | void printCardUID();      // Print detected card UID to serial port
5 | bool checkAuthorization(byte* cardUid); // Verify if card UID is in authorized list
6 | void unlockDoor();       // Door unlock logic

```

[Back to Elegoo](#)

2. Implementation File: RC522_control.cpp

(1) Global Variable Definitions

```

1 | const byte authorizedUIDs[][4] = {
2 |   {0x83, 0xE8, 0x8D, 0x04},
3 |   {0x31, 0x1A, 0xCE, 0x05},
4 |   {0xFB, 0x5A, 0x25, 0x02},
5 | };

```

- List of authorized card UIDs, each UID is 4 bytes (standard MIFARE card)

```

1 | const int AUTHORIZED_COUNT = sizeof(authorizedUIDs) / sizeof(authorizedUIDs[0]);

```

- Number of authorized cards, automatically calculated from array length

```

1 | MFRC522 mfrc522(SS_PIN, RST_PIN); // RC522 module object
2 | MFRC522::MIFARE_Key key;          // MIFARE card key object
3 | Servo doorServo;                  // Servo motor object

```

- Global object instantiation, corresponding to declarations in header file

(2) initRC522()

Function: Initialize RC522 RFID module, including SPI bus and hardware initialization.

```

1 | SPI.begin(); // Initialize SPI bus
2 | pinMode(RST_PIN, INPUT_PULLUP);
3 | mfrc522.PCD_Init(); // Initialize RC522 module hardware

```

- Initialize SPI bus (communication interface between ESP32 and RC522)
- Set RST pin as input with internal pull-up
- Initialize RC522 hardware to enter standby mode for card reading

(3) initServo()

Function: Initialize door lock servo motor, set initial state to locked.

```

1 | doorServo.attach(SERVO_PIN); // Attach servo to the specified GPIO pin
2 | doorServo.write(LOCKED_ANGLE); // Initial state: Locked (set to locked angle)
3 | delay(500); // Wait 500ms for servo to reach target position stably

```

- Attach servo to specified GPIO pin (SERVO_PIN)
- Initial state: Locked (set to locked angle 0 degrees)
- Wait 500ms for servo to stably reach target position

(4) initMifareKey()

Function: Initialize MIFARE card authentication key using default factory key.

```

1 | for(byte i = 0; i < 6; i++){
2 |   key.keyByte[i] = 0xFF; // Assign each byte of the 6-byte key
3 | }

```

- Default factory key for MIFARE cards: 0xFFFFFFFFFFFF
- Key is required for RC522 to communicate securely with MIFARE cards

(5) printCardUID()

Function: Read and print the UID of detected IC card via serial port.

[Back to Elegoo](#)

```
1 | for(byte i = 0; i < mfrc522.uid.size; i++){
2 |   Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? "0" : "");
3 |   Serial.print(mfrc522.uid.uidByte[i], HEX);
4 | }
```

- UID (Unique Identifier) is the unique code of the IC card (4 or 7 bytes)
- Unified format: pad with leading 0 if value < 0x10 for easy debugging

(6) *checkAuthorization(byte cardUid)**

Function: Verify if the current card's UID is in the authorized list.

```
1 | for(int i = 0; i < AUTHORIZED_COUNT; i++){
2 |   if(memcmp(cardUid, authorizedUIDs[i], 4) == 0){
3 |     return true;
4 |   }
5 | }
6 | return false;
```

- Traverse all authorized UIDs, compare byte by byte
- `memcmp` : Efficiently compare 4-byte UID (returns 0 if fully matched)
- Returns true: Authorized card; Returns false: Unauthorized card

(7) *unlockDoor()*

Function: Execute the complete cycle of unlock → delay → auto-lock.

```
1 | doorServo.write(UNLOCKED_ANGLE); // Rotate servo to unlocked angle
2 | delay(UNLOCK_DURATION); // Maintain unlocked state for specified duration
3 | doorServo.write(LOCKED_ANGLE); // Rotate servo back to locked angle
```

- Control servo to rotate to unlocked angle (30 degrees)
- Maintain unlocked state for specified duration (3000ms)
- Rotate servo back to locked angle (0 degrees) to complete access control cycle

3. Header File: `key.h`

Function: Define OLED display configuration, keypad-related function declarations, and global variables.

```
1 | #define SCREEN_WIDTH 128
2 | #define SCREEN_HEIGHT 64
3 | #define OLED_RESET -1 // No reset pin
4 | extern Adafruit_SSD1306 oled;
```

- `SCREEN_WIDTH/SCREEN_HEIGHT` : OLED display resolution (128×64 pixels)
- `OLED_RESET` : Reset pin (-1 means not used)
- `oled` : OLED display object (instantiated in `.cpp`)

Function Declarations:

```
1 | void initKeypadAndOled(); // Initialize keypad + OLED
2 | void getKeypad(); // Read keypad input
3 | void displayInputOnOled(); // Display input digits on OLED
```

Global Variable:

```
1 | extern String inputBuffer; // Global input buffer (stores keypad input digits)
```

4. Implementation File: `key.cpp`

(1) Constant Definitions

[Back to Elegoo](#)

```
1 | const long PASSWORD = 123456; // Preset password
2 | const byte ROWS = 4;
3 | const byte COLS = 4;
4 | char hexaKeys[ROWS][COLS] = {
5 |   {'1','2','3','A'},
6 |   {'4','5','6','B'},
7 |   {'7','8','9','C'},
8 |   {'*','0','#','D'}
9 | };
10 | byte rowPins[ROWS] = {13, 12, 14, 27};
11 | byte colPins[COLS] = {26, 25, 33, 32};
```

- **PASSWORD** : Preset password (123456)
- **ROWS/COLS** : Number of keypad rows/columns (4×4 matrix)
- **hexaKeys** : Keypad key mapping table
- **rowPins/colPins** : Keypad row/column pin definitions

```
1 | Keypad customKeypad = Keypad( makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS);
```

- Create keypad object, associate key mapping and pins

(2) Global Objects

```
1 | Adafruit_SSD1306 oled(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
2 | String inputBuffer = "";
```

- **oled** : OLED display object, using I2C communication
- **inputBuffer** : Global input buffer (only stores digits 0–9)

(3) initKeypadAndOled()

Function: Initialize keypad and OLED display module.

```
1 | Wire.begin(21, 22); // ESP32 hardware I2C pins: SDA=21, SCL=22
```

- Initialize I2C bus, specify SDA and SCL pins

```
1 | if(!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)){
2 |   Serial.println(F("OLED initialization failed!"));
3 |   while(1); // Halt if initialization fails
4 | }
```

- Initialize OLED, use internal charge pump, default I2C address 0x3C
- Halt and output error if initialization fails

```
1 | oled.setTextColor(WHITE); // White font color
2 | oled.setTextSize(2); // Font size (2×)
```

- Set font color to white
- Set font size to 2 (16×16 pixels)

(4) checkPassword()

Function: Password verification function, convert input buffer to integer and compare with preset password.

```
1 | long inputNum = inputBuffer.toInt();
2 | return (inputNum == PASSWORD);
```

- Convert string buffer to integer
- Return whether input matches preset password

[Back to Elegoo](#)

(5) displayPwdResult(bool isMatch)

Function: Display password verification result.

```
1 | if (isMatch) {
2 |   oled.println("Correct!");
3 |   oled.display();
4 |   unlockDoor();
5 | } else {
6 |   oled.println("Error!");
7 |   oled.display();
8 | }
```

- Password correct: Display "Correct!" and trigger door unlock
- Password incorrect: Display "Error!"

```
1 | delay(1500); // Display for 1.5 seconds
2 | inputBuffer = ""; // Clear input buffer
3 | displayInputOnOled(); // Return to input display
```

- Display result for 1.5 seconds
- Clear input buffer
- Return to input display interface

(6) getKeypad()

Function: Read keypad input, handle digit input, password verification, and delete operation.

```
1 | char customKey = customKeypad.getKey();
```

- Get currently pressed key

```
1 | if (customKey >= '0' && customKey <= '9') {
2 |   if (inputBuffer.length() < 6) {
3 |     inputBuffer += customKey;
4 |     displayInputOnOled();
5 |   }
6 | }
```

- Only add digits 0–9 to buffer
- Limit input length to 6 digits (matching preset password length)

```
1 | else if (customKey == '#') {
2 |   if (inputBuffer.length() == 6) {
3 |     bool isCorrect = checkPassword();
4 |     displayPwdResult(isCorrect);
5 |   } else {
6 |     oled.println("Please enter 6 digits");
7 |     delay(1000);
8 |     displayInputOnOled();
9 |   }
10 | }
```

- Press # key: Trigger password verification
- Only verify when 6 digits are entered, otherwise prompt for complete input

```
1 | else if (customKey == '*') {
2 |   if (inputBuffer.length() > 0) {
3 |     inputBuffer.remove(inputBuffer.length() - 1);
4 |     displayInputOnOled();
5 | }
```

```
6 | }  
  | }
```

[Back to Elegoo](#)

- Press * key: Delete last digit (backspace function)
- Only delete when there is input

(7) displayInputOnOled()

Function: Display entered digits on OLED.

```
1 | oled.clearDisplay();  
2 | oled.setTextSize(2);  
3 | oled.setCursor(0, 0);  
4 | oled.println("PASSWORD");  
5 | oled.setCursor(0, 30);  
6 | oled.println(inputBuffer);  
7 | oled.display();
```

- Clear display buffer
- Display "PASSWORD" title at top
- Display entered digits below
- Refresh display to OLED screen

5. Main File: Smart_Access_Control.ino

Global Variables:

```
1 | int buzzer = 5; // the pin of the active buzzer  
2 | #define RFID_CHECK_INTERVAL 100  
3 | unsigned long lastRFIDCheck = 0; // Record last RC522 check time (for frequency limiting)
```

- **buzzer** : Active buzzer pin (GPIO 5)
- **RFID_CHECK_INTERVAL** : RFID check interval (100 milliseconds)
- **lastRFIDCheck** : Last RC522 check time (used for frequency limiting)

setup()

Function: Initialize system hardware and all modules.

```
1 | Serial.begin(9600);  
2 | while (!Serial); //Wait for serial port to initialize
```

- Initialize serial communication, baud rate 9600
- Wait for serial port to finish initialization

```
1 | pinMode(buzzer, OUTPUT); // initialize the buzzer pin as an output
```

- Set buzzer pin as output mode

```
1 | initKeypadAndOled(); // Initialize keypad and OLED  
2 | initRC522(); // Initialize RC522 RFID module (SPI communication)  
3 | initServo(); // Initialize door lock servo motor  
4 | initMifareKey(); // Initialize MIFARE card authentication key
```

- Initialize in sequence: keypad+OLED, RC522, servo, MIFARE key

handleRFID()

Function: Handle RC522 RFID card detection, UID reading, and authorization verification (frequency-limited).

```
1 | if (mfr522.PICC_IsNewCardPresent() && mfr522.PICC_ReadCardSerial()) {
```

- Detect if a new card is present and read its serial number (UID)

[Back to Elegoo](#)

```
1 | printCardUID(); // Print detected card UID via serial port
```

- Print detected card UID via serial port

```
1 | if (mfr522.uid.size == 4) {
2 |   if (checkAuthorization(mfr522.uid.uidByte)) {
3 |     digitalWrite(buzzer, HIGH);
4 |     delay(20);
5 |     digitalWrite(buzzer, LOW);
6 |     delay(20);
7 |     Serial.println(F("RFID Authorization Passed - Unlocking Door!"));
8 |     unlockDoor();
9 |   } else {
10 |    Serial.println(F("RFID Unauthorized Card!"));
11 |   }
12 | }
```

- Only support 4-byte UID (standard MIFARE IC card)
- Verify if card UID is in authorized list
- Authorization passed: Buzzer beeps twice, trigger door unlock
- Unauthorized: Output error message

```
1 | mfr522.PICC_HaltA();
2 | mfr522.PCD_StopCrypto1();
```

- Halt the card and stop cryptographic communication (required for RC522 normal operation)

loop()

Function: Main loop, handle IR keypad priority and RFID frequency-limited detection.

```
1 | getkeypad(); // IR Priority Processing (Ensure Sensitivity)
```

- Read keypad input (priority processing to ensure response sensitivity)

```
1 | unsigned long now = millis();
2 | if (now - lastRFIDCheck >= RFID_CHECK_INTERVAL) {
3 |   lastRFIDCheck = now;
4 |   handleRFID(); // Limit SPI communication frequency to avoid interfering IR
5 | }
```

- Check RC522 every 100ms
- Limit SPI communication frequency to avoid interfering with IR keypad

Code Modification Tips

Modify Preset Password

Modify Location: key.cpp

```
1 | const long PASSWORD = 123456; // Preset password
```

- Modify this value to change the preset password of the access control system
- Password length must match the input limit (currently limited to 6 digits)

Add/Remove Authorized Cards

Modify Location: RC522_control.cpp

```
1 | const byte authorizedUIDs[][4] = {  
2 |   {0x83, 0xE8, 0x8D, 0x04},  
3 |   {0x31, 0x1A, 0xCE, 0x05},  
4 |   {0xFB, 0x5A, 0x25, 0x02},  
5 | };
```

- Add new UID to array to authorize new cards
- Remove UID from array to deauthorize cards
- `AUTHORIZED_COUNT` is automatically calculated, no manual modification needed

Modify Servo Angles

Modify Location: RC522_control.h

```
1 | #define LOCKED_ANGLE 0 // Door locked angle (degrees)  
2 | #define UNLOCKED_ANGLE 30 // Door unlocked angle (degrees)
```

- `LOCKED_ANGLE` : Door locked angle, adjust according to actual mechanical structure
- `UNLOCKED_ANGLE` : Door unlocked angle, typically 30–90 degrees

Modify Unlock Hold Duration

Modify Location: RC522_control.h

```
1 | #define UNLOCK_DURATION 3000 // Unlock hold duration (milliseconds)
```

- Modify this value to adjust the hold duration after unlocking
- Unit: milliseconds, e.g., change to 5000 for 5 seconds hold

Modify RFID Check Frequency

Modify Location: Smart_Access_Control.ino

```
1 | #define RFID_CHECK_INTERVAL 100
```

- Unit: milliseconds, smaller value means more frequent checks
- Decrease this value for faster RFID response, but increases SPI communication load
- Current value 100ms balances response speed and system stability

Modify OLED Display Content

Modify Location: `displayInputOnOled()` in key.cpp

```
1 | oled.setTextSize(2); // Modify font size  
2 | oled.setCursor(0, 0); // Modify display position  
3 | oled.println("PASSWORD"); // Modify title text
```

- Modify `setFontSize()` parameter to adjust font size (1=8×8, 2=16×16)
- Modify `setCursor()` parameter to adjust display position
- Modify `println()` content to customize title text

Modify Buzzer Control

Modify Location: Smart_Access_Control.ino

```
1 | digitalWrite(buzzer, HIGH);  
2 | delay(20);  
3 | digitalWrite(buzzer, LOW);  
4 | delay(20);
```

- Modify `delay()` parameters to adjust buzzer beep duration
- Add more repetitions for multiple beep alerts

- Remove this code block to disable buzzer feedback

[Back to Elegoo](#)

Modify Keypad Input Length Limit

Modify Location: key.cpp

```
1 | if (inputBuffer.length() < 6){
```

- Modify the number 6 to adjust maximum input length
- Must also modify preset password length to maintain match

[← snake game](#)

[over view→](#)